

Sustained 70B AWQ Inference on a Single NVIDIA L20: Throughput, Stability, Energy, and Quality Characterization

Yin Li

<https://github.com/Kevin-Li-2025/llm-quant-bench>

Technical Report Draft: May 20, 2026

Abstract

Serving 70B-class open-weight language models is usually associated with 80GB accelerators, tensor-parallel multi-GPU systems, or vendor-managed inference profiles. This report asks a narrower systems question: can a single NVIDIA L20 48GB GPU sustain a useful 70B-class quantized serving workload, and what are the throughput, latency, stability, energy, and quality boundaries of that configuration? We evaluate Qwen2.5-72B-Instruct-AWQ served with vLLM 0.8.5.post1 and AWQ Marlin on one L20. Under a fixed workload of approximately 512 input tokens and 256 output tokens, a 24-hour concurrency-10 soak completed 36,740/36,740 requests with no request failures and no vLLM CUDA OOM, traceback, or killed-process signatures. The system sustained 108.84 output tokens/s, with p95 time-to-first-token (TTFT) of 6.61s and p95 end-to-end latency of 23.54s. GPU-board power sampled through `nvidia-smi` produced an estimated 7.92 kWh over the run, corresponding to 0.330 output tokens/J and 1.008 total tokens/J. Repeated fixed-shape runs at concurrency 1, 4, 8, and 16 completed 12/12 runs successfully; the concurrency-16 condition averaged 127.22 ± 12.68 output tokens/s over three runs, matching the earlier single-run screening result of 127.70 output tokens/s. The same AWQ endpoint also produced absolute quality scores of 0.8130 on MMLU, 0.8309 on CMMLU, and 0.8082 on GSM8K, plus 80/80 MT-Bench answer generations and a 60-item 8K LongBench subset.

The evidence supports a specific claim: a carefully configured single L20 can serve Qwen2.5-72B-Instruct-AWQ as a throughput-oriented 70B endpoint under the tested fixed-shape workload. It does not prove lossless AWQ quality retention, low-latency interactive serving, broad production SLA coverage, or equivalence to a BF16/FP16 baseline. BF16/FP16 baseline retention and MT-Bench judge scoring remain blocked until external or multi-GPU baseline and judge endpoints are available.

1 Executive Summary

The central result is operational: a single NVIDIA L20 can repeatedly and stably serve a 70B-class AWQ model when the serving profile is designed around short-context, throughput-oriented batching. Table 1 gives the headline measurements.

This report should be read as an empirical systems characterization, not as a new quantization algorithm paper. The contribution is the measurement package: serving configuration, fixed-shape throughput, repeated-run variance, 24-hour stability, GPU-board energy, absolute quality checks, and explicit blockers for stronger quality-retention claims.

Measurement	Result
Model	Qwen2.5-72B-Instruct-AWQ
Runtime	vLLM 0.8.5.post1, AWQ Marlin
GPU	1x NVIDIA L20, 46GB visible VRAM
Fixed-shape workload	approx. 512 input / 256 output tokens
24h soak success	36,740/36,740 requests
24h soak throughput	108.84 output tokens/s
24h soak p95 TTFT / latency	6.61s / 23.54s
24h GPU-board energy	7.92 kWh
24h output tokens/J	0.330
Repeated c16 throughput	127.22 $\pm$ 12.68 output tokens/s
Repeated c16 p95 TTFT / latency	11.91s $\pm$ 0.15s / 34.95s $\pm$ 1.00s
Repeated run success	12/12 runs, 100% success
Quality, absolute AWQ	MMLU 0.8130, CMMLU 0.8309, GSM8K 0.8082
Long-context subset	60/60 LongBench 8K subset requests completed
Explicit non-claim	No BF16/FP16 retention or MT-Bench judge score yet

Table 1: Headline results. Confidence intervals are two-sided 95% intervals over three repeated run-level summaries using Student t critical values.

2 Scope and Non-Claims

The supported claim is:

A single NVIDIA L20 can sustain Qwen2.5-72B-Instruct-AWQ serving under a fixed approximately 512 input / 256 output token workload, with 24-hour stability at concurrency 10 and repeated short-run throughput near 127 output tokens/s at concurrency 16.

The following claims are not supported by the current evidence:

- AWQ is lossless relative to BF16/FP16.
- The configuration is a low-latency chat endpoint.
- The result generalizes to all 70B models, all prompts, or all quantization formats.
- The 8K LongBench subset is an official LongBench leaderboard result.
- MT-Bench has a judge score.
- Single-L20 AWQ generally replaces multi-GPU BF16/FP16 inference.

The non-claims are important. A technical report is only useful if the boundaries are as concrete as the positive result.

3 Introduction

The practical value of 70B-class open-weight language models is clear: they are strong enough for many enterprise and research workloads while remaining self-hostable. The deployment difficulty is that most 70B serving discussions assume 80GB GPUs, multi-GPU tensor parallelism, or

vendor-optimized inference profiles. Many organizations, however, have access to smaller datacenter accelerators such as the NVIDIA L20. A 48GB-class GPU can hold 4-bit 70B weights, but static model fit does not imply sustained serving capacity. KV-cache allocation, prompt shape, output length, batching policy, quantization kernels, thermal behavior, and tail latency determine whether the system is useful.

This report studies one concrete question:

What can one NVIDIA L20 actually sustain when serving a 70B-class AWQ model under fixed, reproducible workloads?

The answer is empirical. We do not propose a new model, quantization method, or serving algorithm. Instead, we characterize a constrained deployment point that is often discussed informally but rarely reported with day-long stability, confidence intervals, energy estimates, and quality checks.

Contributions.

- We report a 24-hour concurrency-10 fixed-shape soak of Qwen2.5-72B-Instruct-AWQ on one NVIDIA L20, including success rate, TTFT, latency, output throughput, request throughput, GPU-board energy, and log-based OOM/error checks.
- We repeat the fixed-shape workload at concurrency 1, 4, 8, and 16 with three runs per condition, reporting run-level means and 95% confidence intervals.
- We report absolute AWQ quality measurements on MMLU, CMMLU, GSM8K, MT-Bench answer generation, and an 8K LongBench subset [17, 1].
- We compare the measured operating point against public Qwen 2.5 72B Q4 serving numbers, Qwen official speed benchmarks, and NVIDIA NIM supported-profile guidance.
- We document the exact blockers for BF16/FP16 baseline retention, MT-Bench judge scoring, runtime ablation, and quantization ablation.

4 Background

4.1 Why 70B on 48GB Is Nontrivial

Weight-only 4-bit quantization can reduce the weight footprint of a 70B-class model enough to fit in a 48GB GPU. Serving still requires more than weights: KV cache, runtime buffers, scheduling state, tokenization overhead, and memory fragmentation all consume capacity. Increasing context length or concurrency can push the same model from stable to OOM. Therefore, “runs on one L20” is not a single claim. It can mean a one-request smoke test, an interactive endpoint, a high-throughput fixed-shape service, or a long-running production workload. This report focuses on the latter two under explicitly bounded shapes.

4.2 vLLM, Continuous Batching, and AWQ Marlin

vLLM is a high-throughput LLM serving engine built around efficient KV-cache management and continuous batching. The PagedAttention work reports large throughput gains from better memory management during LLM serving [6]. AWQ is a post-training weight-only quantization method that

protects salient channels identified through activation statistics [7]. Marlin is an optimized mixed-precision FP16-by-INT4 inference kernel for autoregressive LLM inference [3]. The measured system uses vLLM’s AWQ Marlin path, `awq_marlin`. This matters because a 4-bit checkpoint without an optimized serving kernel is a different deployment configuration.

4.3 Serving Metrics

We follow the serving metric vocabulary used by vLLM and NVIDIA GenAI-Perf: time-to-first-token (TTFT), end-to-end request latency, inter-token latency (ITL), time per output token (TPOT), request throughput, and output token throughput [16, 9]. In a continuously batched system, aggregate server output tokens/s can be much higher than per-request decode speed. This is central to the result: the 24-hour concurrency-10 run produces 108.84 aggregate output tokens/s while individual request decode speed is roughly 11–15 tokens/s.

5 Experimental Setup

5.1 Hardware and Software

Component	Configuration
GPU	1x NVIDIA L20
Visible GPU memory	46,068 MiB reported by <code>nvidia-smi</code>
Model	Qwen/Qwen2.5-72B-Instruct-AWQ
Runtime	vLLM 0.8.5.post1
Quantization path	AWQ Marlin
Serving API	OpenAI-compatible <code>/v1/chat/completions</code>
Primary context profile	<code>max_model_len=1024</code>
Power sampling	<code>nvidia-smi</code> , approximately every 10 seconds

Table 2: Primary test configuration.

The primary service command was:

```
vllm serve /home/USER/models/Qwen2.5-72B-Instruct-AWQ \
--host 0.0.0.0 --port 8001 \
--served-model-name qwen72b-awq-l20 \
--quantization awq_marlin \
--dtype half \
--max-model-len 1024 \
--gpu-memory-utilization 0.98 \
--max-num-seqs 48 \
--max-num-batched-tokens 4096 \
--enforce-eager \
--swap-space 1 \
--disable-log-requests \
--trust-remote-code
```

This is a short-context throughput profile. Separate 4096- and 8192-context single-concurrency checks were run, but the main throughput claim is for the 1024-context service profile.

5.2 Workload

The fixed-shape benchmark was designed to be close to public serving tables:

- 128 unique prompts.
- 498 raw tokenizer prompt tokens.
- Approximately 527 server-side prompt tokens after chat formatting.
- Fixed output length of 256 tokens.
- `max_tokens=256, min_tokens=256, ignore_eos=true, temperature=0`.
- Streaming responses with `stream_options.include_usage=true`.

The 24-hour soak reused the same workload at concurrency 10. Repeated short runs used concurrency 1, 4, 8, and 16 with three repeats per condition.

5.3 Energy and Confidence Intervals

GPU-board energy is computed from sampled board power:

$$E_J = \int P(t) dt, \tag{1}$$

$$\text{output_tok}/J = \frac{\sum_i \text{output_tokens}_i}{E_J}, \tag{2}$$

$$\text{total_tok}/J = \frac{\sum_i (\text{prompt_tokens}_i + \text{output_tokens}_i)}{E_J}. \tag{3}$$

The implementation uses trapezoidal integration over the power trace. These are GPU-board energy estimates, not wall-power measurements.

Repeated-run confidence intervals use run-level summaries:

$$\bar{x} = \frac{1}{n} \sum_{i=1}^n x_i, \tag{4}$$

$$\text{CI}_{95} = \bar{x} \pm t_{0.975, n-1} \frac{s}{\sqrt{n}}. \tag{5}$$

For the reported repeated runs, $n = 3$ per concurrency.

6 Serving Results

6.1 Fixed-Shape Screening Sweep

Table 3 shows the initial fixed-shape concurrency sweep. Throughput improves through concurrency 16 and then regresses at concurrency 24, where tail latency also increases sharply.

Concurrency	Requests	Success	p95 TTFT	p95 Lat.	Out tok/s	Req/s	p05 decode
1	8	100%	0.76s	15.88s	16.21	0.063	16.92
4	32	100%	2.94s	18.63s	57.10	0.223	14.99
8	64	100%	6.06s	22.20s	93.38	0.365	12.13
10	80	100%	6.71s	23.69s	108.70	0.425	11.26
16	128	100%	11.01s	36.29s	127.70	0.499	7.66
24	120	100%	37.26s	64.08s	120.51	0.471	5.25

Table 3: Initial fixed-shape sweep, approximately 512 input tokens and 256 output tokens. Concurrency 16 is the highest-throughput screened point; concurrency 24 is beyond the useful peak for this profile.

Conc.	Runs	Success	Out tok/s	Req/s	p95 TTFT	p95 Lat.
1	3	100% $\pm$ 0.00%	16.57 $\pm$ 1.21	0.065 $\pm$ 0.005	0.54s $\pm$ 1.90s	15.66s $\pm$ 1.94s
4	3	100% $\pm$ 0.00%	55.75 $\pm$ 3.54	0.218 $\pm$ 0.014	3.01s $\pm$ 0.17s	18.70s $\pm$ 0.16s
8	3	100% $\pm$ 0.00%	93.26 $\pm$ 0.23	0.364 $\pm$ 0.001	6.05s $\pm$ 0.06s	22.16s $\pm$ 0.05s
16	3	100% $\pm$ 0.00%	127.22 $\pm$ 12.68	0.497 $\pm$ 0.050	11.91s $\pm$ 0.15s	34.95s $\pm$ 1.00s

Table 4: Repeated fixed-shape runs. All twelve runs completed successfully with no vLLM OOM/error signatures.

6.2 Repeated Fixed-Shape Runs

The c1/c4/c8/c16 conditions were repeated three times each. Table 4 reports run-level means and two-sided 95% confidence intervals.

The repeated concurrency-16 mean, 127.22 output tokens/s, matches the earlier single-run result of 127.70 output tokens/s. The concurrency-8 result is especially tight at 93.26 ± 0.23 output tokens/s. The concurrency-16 throughput interval is wider because two runs produced roughly 130 output tokens/s and one run produced 121.32 output tokens/s. Tail latency remains the main cost of the high-throughput profile.

6.3 Twenty-Four-Hour Soak

Table 5 reports the 24-hour concurrency-10 soak. The run completed normally, and log inspection found zero CUDA OOM, OutOfMemory, Traceback, ERROR, or Killed signatures.

The agreement between the short screening run and the day-long run is one of the strongest results in the report. A short result can be a transient peak; a 24-hour run at the same throughput with zero failures is evidence of sustained stability under this exact workload.

6.4 GPU-Board Energy

The energy result is useful for regression testing and rough deployment planning. It should not be used as a full data-center energy estimate because CPU, memory, fans, power-supply losses, and cooling are outside the measurement scope.

7 Quality Characterization

This report does not yet include BF16/FP16-vs-AWQ retention. It does include absolute quality measurements from the AWQ candidate. These are useful for detecting severe quality collapse and

Metric	Value
Duration	86,412.60s
Total requests	36,740
Successful requests	36,740
Failed requests	0
Success rate	100%
Prompt tokens	19,361,980
Output tokens	9,405,440
Total tokens	28,767,420
Output throughput	108.84 output tok/s
Request throughput	0.425 req/s
p95 TTFT	6.61s
p95 latency	23.54s
p05 per-request decode speed	11.26 tok/s
OOM/error signatures	0

Table 5: Twenty-four-hour fixed-shape soak at concurrency 10. The 24-hour throughput, 108.84 output tokens/s, matches the short concurrency-10 screening result of 108.70 output tokens/s.

Metric	Value
Power samples	8,618
Power trace duration	86,410.0s
Average GPU board power	330.15W
GPU board energy	28,528,468J
GPU board energy	7.92 kWh
Output tokens/J	0.330
Total tokens/J	1.008
Joules/output token	3.033
Joules/total token	0.992

Table 6: GPU-board energy estimate from `nvidia-smi` power samples. This is not a full-system wall-power measurement.

for documenting the measured candidate, but they do not prove that AWQ is lossless.

The three MMLU failures were context-limit validation failures from prompts that exceeded the 1024-token short-context service profile, not CUDA OOMs. The LongBench v1 task-metric post-process scored the 60 generated samples with task-specific metrics: 38.67% on `multifieldqa_en`, 30.53% on `hotpotqa`, 12.90% on `multi_news`, 12.08% on `gov_report`, 2.76% on `lcc`, and 0.00% on `passage_count`. These numbers are stricter than the earlier single-token-F1 summary, but they remain a subset result.

7.1 Blocked Retention and Judge Results

We ran a preflight for the two missing quality claims. Table 8 summarizes the result.

The remote host currently stages only AWQ checkpoint directories under `/home/USER/models`. A local 72B BF16/FP16 baseline is not feasible on one L20: 72B BF16/FP16 weights require roughly 144GB before KV cache and runtime overhead, while the host exposes about 46GB of GPU memory and about 63GB of free disk. We also intentionally avoid judging MT-Bench with the AWQ candidate itself, because self-judging would not be a valid MT-Bench judge score.

Evaluation	Context	Items	OK	Failed	Score
MMLU	1024	14,042	14,039	3	0.8130
CMMLU	1024	11,582	11,582	0	0.8309
GSM8K	1024	1,319	1,319	0	0.8082
MT-Bench generation	1024	80	80	0	pending judge
LongBench subset, token-F1	8192	60	60	0	0.2038
LongBench v1 task metrics	8192	60	60	0	16.16%

Table 7: Absolute AWQ candidate quality measurements. MT-Bench currently reports answer generation only, not a judge score. LongBench is a 60-item 8K subset, not an official leaderboard run.

Item	Status	Missing prerequisites
BF16/FP16 baseline retention	Blocked	<code>BASELINE_BASE_URL</code> , <code>BASELINE_MODEL</code>
MT-Bench judge score	Blocked	<code>JUDGE_BASE_URL</code> , <code>JUDGE_MODEL</code> , <code>JUDGE_API_KEY</code>

Table 8: Quality-retention and MT-Bench judge blockers as of the latest preflight.

8 Long-Context Capacity Checks

Context	Prompt tokens	Concurrency	Success	p95 TTFT	Out tok/s	OOM
4096	3,875	1	5/5	5.28s	10.02	No
8192	7,514	1	3/3	11.03s	6.16	No

Table 9: Single-concurrency long-context capacity checks. These are not high-throughput long-context serving results.

The 4K and 8K checks show that the model can handle longer prompts at concurrency 1 without OOM. They do not justify a high-concurrency long-context claim. Long-context throughput requires a separate service profile, workload matrix, and soak.

9 External Positioning

Table 10 compares the result with public numbers. These comparisons are directional because serving throughput depends on model variant, quantization format, kernel, scheduler, prompt length, output length, concurrency, sampling settings, and measurement method.

The closest public table is GigaGPU’s April 2026 benchmark, which reports 512-token prompts, 256-token generations, and 10 concurrent users for Qwen 2.5 72B Q4 [5]. Under that shape, this single-L20 AWQ Marlin result is above the published single-GPU figures in that table. This does not prove that L20 is generally faster than those GPUs; it shows that this specific vLLM/AWQ-Marlin configuration is competitive under the tested workload.

The model itself is not new. Qwen2.5-72B-Instruct is documented by the Qwen team’s model card and technical report [12, 15]. Qwen’s official speed benchmark is useful context for batch-1 and long-output operating points, but it measures different shapes from the aggregate serving workloads studied here [13].

NVIDIA NIM’s supported-model table is also relevant. Its optimized Qwen2.5 72B Instruct FP8 L20 profiles list 4 or 8 L20 GPUs, while A100 SXM BF16 profiles also use multi-GPU configurations

Source	Hardware	Model / Quant	Shape	Result
This work	1x L20 48GB	Qwen2.5-72B AWQ Marlin	512/256, c10, 24h	108.84
This work	1x L20 48GB	Qwen2.5-72B AWQ Marlin	512/256, c16, repeated	127.22 $\pm$ 12.68
GigaGPU Apr. 2026	1x RTX 3090	Qwen 2.5 72B Q4	512/256, c10	32
GigaGPU Apr. 2026	1x RTX 5090	Qwen 2.5 72B Q4	512/256, c10	58–82
GigaGPU Apr. 2026	1x RTX 6000 Pro	Qwen 2.5 72B Q4	512/256, c10	45
Qwen official	1x A100 80GB	Qwen2.5-72B AWQ, Transformers	input 1, output 2048, batch 1	11.50
Qwen official	2x A100 80GB	Qwen2.5-72B AWQ, vLLM	input 1, output 2048, batch 1	44.30
Qwen official	2x A100 80GB	Qwen2.5-72B AWQ, vLLM	input 30720, output 2048, batch 1	30.02

Table 10: Directional comparison. The GigaGPU rows are the closest public 512-input/256-output/10-concurrent shape. Qwen official rows are batch-1 speed references and should not be directly compared to aggregate concurrency-10 or concurrency-16 throughput.

depending on profile [10]. Our result demonstrates a different deployment point: single-L20 AWQ Marlin outside that conservative FP8 profile matrix.

10 Discussion

10.1 The Result Is Throughput-Oriented

The service is not a low-latency chat profile. At concurrency 16, the repeated run mean p95 TTFT is 11.91s and p95 end-to-end latency is 34.95s for fixed 256-token outputs. This is useful for batch generation, asynchronous agents, offline summarization, internal processing, and throughput-oriented serving. It is not ideal for interactive applications that require sub-second first tokens.

10.2 Why Repetition Matters

The repeated runs strengthen the report because they show that the screening numbers were not one-off peaks. Concurrency 8 is highly stable across three runs. Concurrency 16 has wider throughput variance, but the mean remains close to the initial single-run result and all runs complete successfully. The 24-hour soak provides a different kind of evidence: long-run operational stability at the more balanced concurrency-10 point.

10.3 What Would Make This a Stronger Research Paper

The report becomes closer to a full research paper if it answers comparative questions rather than only characterizing one deployment point. The highest priority additions are:

- BF16/FP16 baseline quality retention using the same prompts, datasets, decoding settings, and scorers.
- MT-Bench judge scoring with an external judge model and answer-order swapping.
- Runtime ablation: vLLM versus SGLang versus llama.cpp/GGUF.
- Quantization ablation: AWQ versus GPTQ versus FP8 where compatible.

- Multiple models: Llama 70B, DeepSeek distill 70B, and Qwen3-class 70B checkpoints where available.

11 Reporting Checklist Against Strong Technical Reports

Strong modern technical reports tend to combine a narrow central claim, complete experimental context, broad evaluations, clear failure analysis, and explicit release artifacts. GPT-4 emphasizes capability, limitation, and safety evaluation while withholding some architecture details [11]. Llama 3 and Gemini 1.5 provide broad evaluation suites across model variants and modalities [8, 4]. DeepSeek-V3 highlights architecture, training stability, cost, and inference efficiency [2]. MLPerf Inference is not an LLM model report, but it sets a useful benchmark-reporting standard: fixed scenarios, quality targets, latency bounds, submission review, and reproducible result artifacts [14].

Table 11 audits this report against that standard. The purpose is to prevent overclaiming and to make the remaining work obvious.

Dimension	Strong-report expectation	Current status	Upgrade needed
Claim framing	One central claim with explicit scope and non-claims	Strong: supported claim and non-claims are stated early	Keep the claim narrow in title, abstract, and README
Method details	Hardware, software, workload, decoding, and metrics	Strong for the measured vLLM/AWQ setup	Add driver/CUDA exact versions and wall-power instrumentation
Statistical rigor	Repeated runs, uncertainty, and tail metrics	Moderate: 3 repeats per short condition plus 24h soak	Run more repeats across days or machines
Quality evidence	Matched baselines and benchmark-specific scoring	Partial: MMLU/CMMLU/GSM8K absolute scores, MT-Bench generation, LongBench subset	Add BF16/FP16 retention, MT-Bench judge, and official LongBench runs
Comparative evidence	Controlled ablations against alternatives	Weak: public directional comparisons only	Add vLLM/SGLang/llama.cpp and AWQ/GPTQ/FP8 ablations
Failure analysis	OOM, errors, tails, and boundary conditions	Moderate: OOM/error scan and c24 tail-latency regression are reported	Add explicit failure-point sweeps for context and concurrency
Energy/cost	Measurement scope and cost model	Partial: GPU-board energy only	Measure wall power and convert to cost/token
Artifacts	Code, configs, raw outputs, and source package	Strong: repo, commands, generated PDF, and arXiv source package	Publish raw logs/checksums or a reproducible artifact archive
Narrative/figures	A few plots that make the result visually obvious	Weak: mostly tables	Add throughput-latency curves, power trace, and quality summary figures

Table 11: Self-audit against common expectations in high-quality technical reports and benchmark reports.

Overall, the current document is credible as a narrow technical report. It is not yet a top-tier research paper because the strongest missing evidence is comparative rather than editorial: controlled full-precision retention, official judge scoring, runtime ablations, quantization ablations, and broader replication.

12 Threats to Validity

Single model and quantization path. The main results use Qwen2.5-72B-Instruct-AWQ with AWQ Marlin. Other models, quantization formats, and kernels may behave differently.

Fixed workload. The primary workload has approximately 512 input tokens and fixed 256 output tokens. Real traffic varies in prompt length, output length, retry behavior, tool calls, and burstiness.

Short-context throughput profile. The highest-throughput results use `max_model_len=1024`. They should not be generalized to high-concurrency 8K, 16K, or 32K serving.

No BF16/FP16 retention yet. Absolute AWQ quality scores do not prove retention against full precision. A matched baseline endpoint is required.

MT-Bench generation is not judge scoring. The MT-Bench run generated 80/80 answers, but no external judge score is available.

Energy scope. Energy is GPU-board energy from `nvidia-smi`, not wall energy.

Small repeated-run sample. The repeated fixed-shape CI uses three runs per condition. This is enough to catch gross instability, but broader statistical confidence would require more runs across days, hosts, driver versions, and ambient conditions.

13 Reproducibility

The benchmark code and documentation are available at:

<https://github.com/Kevin-Li-2025/llm-quant-bench>

Key remote artifact paths:

- 24h soak: `/home/USER/llm-quant-bench/runs/qwen72b-awq-120/soak-24h-fixed512x256-c10-20260519T034856Z`
- Repeated fixed-shape CI: `/home/USER/llm-quant-bench/runs/qwen72b-awq-120/repeated-fixed512x256-vllm-awq-20260520T120138Z`
- Quality evaluation: `/home/USER/llm-quant-bench/runs/qwen72b-awq-120/quality-eval-20260520T084323Z`
- BF16/MT-Bench preflight: `/home/USER/llm-quant-bench/runs/qwen72b-awq-120/bf16-mtbench-preflight-20260520T123916Z`

Representative load command:

```
python3 -m llm_quant_bench load \
  --config runs/qwen72b-awq-l20/config.fixed512x256.json \
  --dataset runs/qwen72b-awq-l20/fixed-512in-256out.jsonl \
  --out runs/qwen72b-awq-l20/soak-24h-fixed512x256-c10-.../load \
  --concurrency 10 \
  --duration-seconds 86400 \
  --stream-usage
```

Representative repeated-run command:

```
python3 scripts/run_repeated_load.py \
  --config runs/qwen72b-awq-l20/config.fixed512x256.json \
  --dataset runs/qwen72b-awq-l20/fixed-512in-256out.jsonl \
  --out runs/qwen72b-awq-l20/repeated-fixed512x256-vllm-awq \
  --concurrencies 1 4 8 16 \
  --repeats 3 \
  --duration-seconds 120 \
  --stream-usage
```

14 Conclusion

This report shows that a single NVIDIA L20 can sustain Qwen2.5-72B-Instruct-AWQ serving under a fixed short-context throughput workload. The strongest evidence is the combination of a 24-hour concurrency-10 soak at 108.84 output tokens/s, 12/12 successful repeated fixed-shape runs, and a repeated concurrency-16 mean of 127.22 ± 12.68 output tokens/s. The result is meaningful because it documents a practical single-accelerator deployment point for a 70B-class model with explicit latency, energy, quality, and limitation boundaries.

The correct interpretation is narrow but useful: single-L20 70B AWQ serving is feasible and stable for a controlled throughput-oriented workload. Stronger claims about lossless quality, full-precision retention, judge-scored instruction following, long-context high-concurrency serving, or general production replacement require additional baseline and ablation experiments.

References

- [1] Yushi Bai, Xin Lv, Jiajie Zhang, Hongchang Lyu, Jiankai Tang, Zhidian Huang, Zhengxiao Du, Xiao Liu, Aohan Zeng, Lei Hou, Yuxiao Dong, Jie Tang, and Juanzi Li. Longbench: A bilingual, multitask benchmark for long context understanding. *arXiv preprint arXiv:2308.14508*, 2023.
- [2] DeepSeek-AI. Deepseek-v3 technical report. *arXiv preprint arXiv:2412.19437*, 2024.
- [3] Elias Frantar, Saleh Ashkboos, Torsten Hoefler, and Dan Alistarh. Marlin: Mixed-precision auto-regressive parallel inference on large language models. *arXiv preprint arXiv:2408.11743*, 2024.
- [4] Gemini Team, Google. Gemini 1.5: Unlocking multimodal understanding across millions of tokens of context. *arXiv preprint arXiv:2403.05530*, 2024.

- [5] GigaGPU. Tokens/sec benchmark update, april 2026. <https://gigagpu.com/tokens-sec-benchmark-update-april-2026/>, 2026. Accessed 2026-05-20.
- [6] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with pagedattention. *Proceedings of the ACM SIGOPS 29th Symposium on Operating Systems Principles*, 2023.
- [7] Ji Lin, Jiaming Tang, Haotian Tang, Shang Yang, Xingyu Chen, and Song Han. Awq: Activation-aware weight quantization for llm compression and acceleration. *Proceedings of Machine Learning and Systems*, 2024.
- [8] Llama Team, AI @ Meta. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024.
- [9] NVIDIA. Nvidia genai-perf: Metrics and benchmarking documentation. https://docs.nvidia.com/deeplearning/triton-inference-server/archives/triton-inference-server-2500/user-guide/docs/perf_analyzer/genai-perf/README.html, 2025. Accessed 2026-05-20.
- [10] NVIDIA. Nvidia nim for large language models: Supported models. <https://docs.nvidia.com/nim/large-language-models/1.14.0/supported-models.html>, 2026. Accessed 2026-05-20.
- [11] OpenAI. Gpt-4 technical report. Technical report, OpenAI, 2023.
- [12] Qwen Team. Qwen2.5-72b-instruct model card. <https://huggingface.co/Qwen/Qwen2.5-72B-Instruct>, 2024. Accessed 2026-05-20.
- [13] Qwen Team. Qwen2.5 speed benchmark. https://qwen.readthedocs.io/en/v2.5/benchmark/speed_benchmark.html, 2024. Accessed 2026-05-20.
- [14] Vijay Janapa Reddi, Christine Cheng, David Kanter, Peter Mattson, Guenther Schmuelling, Carole-Jean Wu, Brian Anderson, Maximilien Breughe, Mark Charlebois, William Chou, et al. Mlperf inference benchmark. *arXiv preprint arXiv:1911.02549*, 2019.
- [15] Qwen Team. Qwen2.5 technical report. *arXiv preprint arXiv:2412.15115*, 2024.
- [16] vLLM Project. vllm serving benchmark documentation. <https://docs.vllm.ai/en/latest/cli/bench/serve.html>, 2026. Accessed 2026-05-20.
- [17] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. Judging llm-as-a-judge with mt-bench and chatbot arena. *Advances in Neural Information Processing Systems*, 2023.